

RouteRider Tire Pressure

Living Product & Build Document

Version: 0.4 Draft

Status: 🟡 In Progress

Purpose: Living source of truth for product definition, calculation logic, route analysis, UX and technical implementation

Next milestone: First working MVP with Claude Code

1. Product Vision

RouteRider soll langfristig ein cyclist-first Tool für Bikepacking und Adventure Cycling werden.

Die langfristige Vision verbindet:

Route → Terrain → Surface Conditions → Weather → Bike Setup → Tire Pressure → POIs → Water → Fueling

Für die erste Version wird RouteRider bewusst auf ein einziges Problem reduziert:

Welchen Reifendruck sollte ich für genau diese Route und genau mein Bike-Setup fahren?

Der Nutzer lädt eine GPX-Route hoch. RouteRider analysiert den tatsächlichen Untergrund der Route, kombiniert diese Informationen mit dem individuellen Bike-Setup und berechnet daraus einen empfohlenen Reifendruck für Vorder- und Hinterrad.

Der Tire Pressure Calculator ist damit der erste eigenständig funktionierende Baustein der größeren RouteRider-Vision.

2. Core User Problem

Aktuelle Reifendruck-Rechner berücksichtigen typischerweise Faktoren wie:

- Fahrergewicht
- Fahrradgewicht
- Reifenbreite
- Tubeless oder Tube
- teilweise eine manuell gewählte Terrain-Kategorie

Das Problem:

Der Nutzer muss selbst einschätzen, welchen Untergrund seine geplante Route enthält.

Bei einer längeren Gravel- oder Bikepacking-Route kann diese Einschätzung schwierig sein.

Eine Route kann beispielsweise bestehen aus:

- 35 % Asphalt
- 50 % Gravel
- 15 % unbefestigten Wegen

RouteRider soll diesen Terrain-Mix automatisch aus der Route ableiten und in die Reifendruckempfehlung einbeziehen.

Langfristig soll RouteRider zusätzlich berücksichtigen, dass sich der Zustand desselben Untergrunds durch Wetter verändern kann.

3. Product Hypothesis

Wenn RouteRider den tatsächlichen Untergrund einer geplanten Route analysiert und mit dem individuellen Bike-Setup kombiniert, kann eine stärker auf die konkrete Fahrt zugeschnittene Reifendruckempfehlung entstehen als bei einem Rechner, bei dem der Nutzer das Terrain ausschließlich selbst auswählen muss.

Diese Aussage ist zunächst eine **Produkthypothese**.

Der MVP soll nicht behaupten, objektiv präziser als etablierte Rechner wie SILCA zu sein.

Die Genauigkeit soll iterativ durch:

- Vergleich mit bestehenden Rechnern
- eigene Testfahrten
- reale Nutzerwerte
- spätere Surface- und Weather-Daten

validiert und verbessert werden.

4. MVP Goal

Der erste MVP ermöglicht:

GPX Upload

↓

Route Analysis

↓

OSM Surface Analysis



Surface Confidence



Bike Setup



Pressure Calculation



Recommendation

Ohne Account, Login oder zusätzliche Einrichtung.

5. Step 1 — Route

Der Nutzer lädt eine `.gpx` Datei hoch.

Unterstützt werden:

- File Picker
- Drag & Drop
- Mobile File Selection

RouteRider extrahiert aus der GPX-Datei:

- Koordinaten
- Route Geometry
- Distanz
- Elevation, sofern vorhanden und ausreichend zuverlässig

Nach erfolgreichem Upload zeigt RouteRider:

- Route auf Karte
- Dateiname
- Distanz
- Höhenmeter, soweit zuverlässig

CTA:

Analyze Route

6. Important GPX Principle

● Decision

Eine GPX-Datei enthält grundsätzlich nicht automatisch die benötigten OpenStreetMap-Surface-Daten.

Sie liefert primär eine geografische Route aus Koordinaten.

RouteRider muss deshalb:

```text id="upatme" GPX ↓ Coordinates / Route Geometry ↓ Map Matching ↓ Matched OSM Ways ↓  
OSM Attributes ↓ Surface Classification

durchführen.

Die GPX-Datei und die OSM-Daten sind damit zwei getrennte Datenquellen, die RouteRider miteinander verbindet.

---

### # 7. Map Matching

#### ● **\*\*Critical Technical Requirement\*\***

Die Zuordnung einer GPX-Route zu den korrekten OpenStreetMap Ways ist ein zentraler technischer Bestandteil des Produkts.

RouteRider darf nicht ausschließlich:

> Nearest GPX Point → Nearest OSM Way

verwenden.

GPS-Ungenauigkeiten, parallele Straßen, Radwege, Kreuzungen und dicht nebeneinander liegende Trails können ansonsten zu falschen Surface-Zuordnungen führen.

Ein robustes Matching sollte nach Möglichkeit berücksichtigen:

- Distance
- Geometry
- Direction
- Geometry Overlap
- Route Continuity

Konzeptionell:

```text id="e54xjl"  
GPX Geometry

```
↓  
Candidate OSM Ways  
↓  
Evaluate Candidates  
↓  
Distance  
+ Direction  
+ Geometry Overlap  
+ Continuity  
↓  
Best Matching OSM Way
```

8. Map Matching Technology

● Open Technical Decision

Vor Implementierung muss geprüft werden, ob RouteRider:

Option A — Own Matching

OSM Ways über Overpass abrufen und selbst gegen die GPX-Geometrie matched.

oder

Option B — Existing Map Matching Service

einen bestehenden Map-Matching-Service nutzt.

Mögliche Technologien sollen vor dem Build verglichen werden.

Ein Kandidat ist beispielsweise:

Valhalla Map Matching

Die Entscheidung soll insbesondere berücksichtigen:

- Matching Accuracy
- OSM Compatibility
- API Availability
- Cost
- Rate Limits
- Complexity
- Scalability
- Möglichkeit, OSM Way Attributes anschließend auszulesen

Für den MVP darf eine einfachere Lösung verwendet werden, solange die Architektur einen späteren Austausch ermöglicht.

9. Step 2 — Surface Analysis

Nach dem Map Matching liest RouteRider verfügbare OSM-Informationen der zugeordneten Ways aus.

Die Strecke wird für den MVP in vier Kategorien eingeteilt:

Paved

Asphaltierte oder befestigte Oberfläche.

Gravel

Schotter und semi-befestigte Wege.

Trail / Unpaved

Erde, Sand, Gras, Trail und andere unbefestigte Oberflächen.

Unknown

Segmente, für die keine ausreichend zuverlässige Klassifizierung möglich ist.

10. OSM Attributes

RouteRider soll nicht ausschließlich `surface=*` abrufen.

Wo verfügbar sollen mindestens folgende OSM-Attribute verarbeitet bzw. gespeichert werden:

```
```text id="0cb9zt" surface smoothness tracktype highway
```

```
V0.1 Calculation
```

Für die eigentliche Pressure Calculation steht zunächst primär:

```
`surface`
```

im Vordergrund.

```
Future
```

Die zusätzlichen Daten ermöglichen später eine differenziertere Einschätzung der tatsächlichen Surface Roughness.

Beispiel:

```
```text id="q5c8jy"
surface=gravel
```

```
smoothness=good
tracktype=grade1
```

kann langfristig anders behandelt werden als:

```
```text id="bgk1dq" surface=gravel smoothness=bad tracktype=grade3
```

Obwohl beide aktuell unter:

**\*\*Gravel\*\***

fallen.

---

## # 11. OSM Surface Classification

### ## Paved

- asphalt
- concrete
- concrete:plates
- concrete:lanes
- paved
- paving\_stones
- sett
- chipseal

### ## Gravel / Semi-Paved

- gravel
- compacted
- fine\_gravel
- unpaved
- crushed\_shell
- pebblestone

### ## Trail / Unpaved

- dirt
- earth
- ground
- grass
- sand
- mud
- rock
- woodchips

### ## Unknown

Alle nicht ausreichend klassifizierbaren Segmente.

### ### Decisions

- `compacted` wird für MVP und frühe Versionen als Gravel behandelt.
- Smooth Gravel / Rough Gravel wird später differenziert.
- Für MVP und frühe Versionen macht diese Differenzierung keinen Unterschied.

---

### # 12. Missing Surface Data

- **\*\*Architecture Principle\*\***

Fehlendes `surface=\*` bedeutet nicht automatisch, dass ein Streckenabschnitt vollständig unbekannt ist.

RouteRider soll eine abgestufte Klassifizierung ermöglichen.

Konzeptionell:

```
```text id="2phd7r"
surface available?
↓ YES
Direct Classification
```

↓ NO

```
tracktype available?
↓ YES
Possible Inference
```

↓ NO

```
highway / other tags sufficiently informative?
↓ YES
Possible Lower-Confidence Inference
```

↓ NO

UNKNOWN

Dabei muss intern unterschieden werden zwischen:

Direct Surface Data

Beispiel:

```
```text id="tgpgpu" surface=gravel
```

### Inferred Surface Data

Beispiel:

```
```text id="ww4f5s"  
highway=track  
tracktype=grade2  
surface=unknown
```

Unknown

Keine ausreichend belastbare Zuordnung.

Die konkreten Inference Rules müssen vor bzw. während der Surface-Implementation definiert und getestet werden.

13. Surface Confidence

RouteRider soll nicht nur anzeigen, wie viel der Strecke klassifiziert wurde.

Langfristig soll Confidence auch berücksichtigen, **wie** die Klassifizierung zustande gekommen ist.

Für V0.1 gilt zunächst:

- **High Confidence:** 80–100 %
- **Medium Confidence:** 60–79 %
- **Low Confidence:** <60 %

● **Decision:** Eine Warnung erscheint ausschließlich bei **Low Confidence**.

Beispiel:

Parts of this route don't have sufficient surface data. Your recommendation may be less accurate.

Future Confidence Model

Später kann zusätzlich unterschieden werden zwischen:

- Direct OSM Surface
- Inferred Surface

- Ambiguous Map Match
- Unknown

Dadurch könnte eine Route mit 95 % klassifizierter Strecke trotzdem eine niedrigere Confidence erhalten, wenn ein großer Teil nur durch schwache Inference bestimmt wurde.

14. Manual Surface Correction

Nach der automatischen Analyse kann der Nutzer die Terrain-Verteilung verändern.

Beispiel:

We analyzed your route

Paved — 42 %

Gravel — 43 %

Trail — 7 %

Unknown — 8 %

Edit Surface Mix

Damit bleibt RouteRider auch bei unvollständigen OSM-Daten nutzbar.

15. Unknown Surface Handling

● Decision

Kombination aus Normalisierung und manueller Korrektur.

Bei High oder Medium Confidence werden bekannte Surface-Anteile für die Pressure Calculation auf 100 % normalisiert.

Beispiel:

```text id="v8hw9u" Original:

Paved 30 % Gravel 40 % Trail 10 % Unknown 20 %

wird für die Berechnung zu:

```
```text id="pv7p0j"
Paved    37.5 %
```

Gravel	50.0 %
Trail	12.5 %

Die ursprüngliche Coverage von 80 % bleibt sichtbar.

Bei Low Confidence wird zusätzlich gewarnt und eine manuelle Korrektur angeboten.

16. Step 3 — Bike Setup

Mandatory Inputs:

Rider Weight

kg

Bike Weight

kg

Luggage Weight

kg

System Weight

Automatisch:

```text id="l46jcd" System Weight = Rider Weight + Bike Weight + Luggage Weight

---

# 17. Tire Setup

### Tire Width

Input in:

`mm`

Die technische Architektur unterstützt:

- Front Tire Width
- Rear Tire Width

### Tire Type

- Tubeless

- Tube

Hinweis:

> Actual measured tire width provides the most accurate result.

---

## # 18. Ride Type & Weight Distribution

● **\*\*Decision\*\***

Weight Distribution ist mandatory.

Der Nutzer wählt einen Ride Type und RouteRider schlägt eine Front/Rear-Verteilung vor.

| Ride Type   Front   Rear           |
|------------------------------------|
| --- ---: ---:                      |
| Performance / Race   45 %   55 %   |
| Gravel / Endurance   47 %   53 %   |
| Bikepacking / Loaded   40 %   60 % |

Diese Werte sind **\*\*Starting Points\*\*** und können verändert werden.

Insbesondere beim Bikepacking hängt die reale Verteilung stark von der Gepäckposition ab.

Hinweis:

> Suggested starting point. Adjust if most of your luggage is carried on the front or rear of the bike.

## ### Future

Spätere Berechnung anhand von:

- Luggage Weight
- Luggage Position
- Bike Geometry
- Rider Position

---

## # 19. Tire Pressure Engine

Die Pressure Engine ist vollständig von der UI getrennt.

```text id="0p7fyb"

Route Data

+

```
Bike Setup
↓
TirePressureEngine
↓
Front Pressure
Rear Pressure
```

20. Base Pressure

● Decision

RouteRider V0.1 verwendet die Berto-basierte Tire-Drop-Formel:

```
```text id="d8o0fp" P (psi) = 600 × L / W2 + 0.75 × W – 25
```

```
`L = individual wheel load in lbs`
```

```
`W = measured tire width in mm`
```

Front und Rear werden unabhängig berechnet.

Die Engine bleibt modular und später kalibrierbar bzw. austauschbar.

---

```
21. Terrain Modifiers
```

```
● **Initial V0.1 Decision**
```

```
```text id="dy8ft4"
```

```
PAVED_MODIFIER = 1.00
```

```
GRAVEL_MODIFIER = 0.90
```

```
TRAIL_MODIFIER = 0.80
```

Damit:

- Paved → Base Pressure
- Gravel → ca. 10 % Reduction
- Trail → ca. 20 % Reduction

Diese Werte sind **Calibration Parameters**, keine unveränderlichen physikalischen Konstanten.

22. Distance-Weighted Terrain Calculation

RouteRider berücksichtigt den tatsächlichen Surface Mix.

Beispiel:

```text id="k3k42z" 40 % Paved 50 % Gravel 10 % Trail

```
```text id="esiy8p"
terrainModifier =
(0.40 × 1.00)
+
(0.50 × 0.90)
+
(0.10 × 0.80)

= 0.93
```

Base Pressure × .

Alle Modifier werden zentral konfiguriert.

23. Tubeless Adjustment

● Open Decision

Noch zu klären:

- Hat Tubeless einen direkten Pressure Modifier?
- Wie groß wäre dieser?
- Oder beeinflusst Tubeless primär den sicheren nutzbaren Pressure Range?

Bis zur Entscheidung wird kein willkürlicher Tubeless Modifier implementiert.

24. Safety Bounds

● New Critical Requirement

RouteRider darf eine mathematisch berechnete Empfehlung nicht automatisch als sicher darstellen.

Reifen, Felgen und Setups können:

- Minimum Pressure
- Maximum Pressure

- Tubeless-specific Limits
- Hookless-specific Limits

besitzen.

Vor öffentlicher Nutzung muss definiert werden, wie RouteRider mit diesen Grenzen umgeht.

Eine berechnete Empfehlung außerhalb eines sicheren Bereichs darf nicht einfach ausgegeben werden.

Open Decision

● Wie erhält RouteRider verlässliche Min/Max Pressure Limits?

Mögliche Ansätze:

- User Input
- Tire Manufacturer Data
- Rim Manufacturer Data
- Kombination verschiedener Datenquellen

25. Pressure Model Limitations

Folgende Faktoren werden aktuell noch nicht vollständig berücksichtigt:

- Tire Casing
- Rim Internal Width
- Exact Tire Construction
- Rider Position
- Detailed Luggage Position
- Surface Roughness
- Weather
- Rider Preference
- Riding Speed

Deshalb wird die Empfehlung als **Starting Pressure** und nicht als absolute optimale Wahrheit kommuniziert.

26. Weather-Aware Surface Conditions

● **Future Product Direction**

Wetter soll langfristig in RouteRider berücksichtigt werden.

Dabei gilt ausdrücklich nicht:

```text id="5e5y5h" Rain = fixed pressure reduction

Stattdessen:

```
```text id="dlhj4f"
```

Weather

↓

Expected Surface Condition

↓

Impact depends on Surface

↓

Pressure Adjustment

Der Einfluss von Wetter ist damit **surface-dependent**.

27. Precipitation

Besonders relevant ist Niederschlag.

Dabei soll langfristig nicht nur berücksichtigt werden:

Current Precipitation

sondern auch:

Recent Precipitation History

Beispielsweise:

Eine Fahrt findet bei trockenem Wetter statt.

In den vorherigen 24 Stunden sind jedoch große Niederschlagsmengen gefallen.

Ein:

surface=ground

Waldweg kann deshalb weiterhin nass, weich oder schlammig sein.

28. Weather + Surface Model

Langfristig:

```
```text id="ulwcrp" OSM Surface + Smoothness / Tracktype + Current Weather + Recent Precipitation ↓  
Expected Surface Condition ↓ Dry Damp Wet Muddy ↓ Pressure Adjustment
```

Der Einfluss unterscheidet sich nach Surface.

Beispielsweise kann Niederschlag bei Asphalt deutlich weniger relevant für die Pressure Recommendation sein als bei:

- Dirt
- Earth
- Ground
- Grass
- Mud
- Trail

---

#### # 29. Weather MVP Decision

● **\*\*Decision\*\***

Weather ist **\*\*nicht Bestandteil von V0.1\*\***.

Grund:

Zunächst muss die Kernkette:

```
```text id="vrfl0"
GPX
↓
Surface
↓
Pressure
```

validiert werden.

Erst danach wird:

```
```text id="ck7vhd" Weather ↓ Surface Condition ↓ Pressure
```

ergänzt.

Dadurch können Fehler und Calibration-Probleme einzelnen Faktoren zugeordnet werden.

---

#### # 30. Future Surface Model

Spätere Versionen können unterscheiden:

- Smooth Paved

- Rough Paved
- Hardpack
- Smooth Gravel
- Rough Gravel
- Chunky Gravel
- Dry Trail
- Wet Trail
- Mud
- Sand

Die vorhandenen OSM-Tags:

```
```text id="tf3npo"
surface
smoothness
tracktype
highway
```

sollen bereits in der Architektur berücksichtigt werden, damit diese Erweiterung keinen vollständigen Umbau benötigt.

31. Future Route Optimization Modes

Später denkbar:

Balanced Route

Pressure optimized across the complete distance-weighted Surface Mix.

Rough Section Priority

Pressure biased toward significant rough sections.

Beispiel:

```
```text id="xb0ahj" 90 % Asphalt 10 % Rocky Trail
```

Ein reiner Durchschnitt kann die Bedeutung der kritischen 10 % unterschätzen.

Nicht Teil des MVP.

---

### # 32. Calibration Strategy

```
```text id="g5ctsl"
RouteRider Experimental Model
```

↓
Comparison with Established Calculators
↓
Own Ride Tests
↓
Calibration
↓
Additional User Data
↓
Improved RouteRider Model

33. Calibration Dataset

Dokumentiert werden:

- System Weight
- Tire Width
- Tube / Tubeless
- Surface Mix
- Surface Confidence
- Ride Type
- Weight Distribution
- RouteRider Front
- RouteRider Rear
- External Calculator Front
- External Calculator Rear
- Actually Ridden Pressure
- Rider Feedback
- Grip
- Comfort
- Bottom-Out / Impact Problems

Später zusätzlich:

- Smoothness
 - Tracktype
 - Weather
 - Previous Rainfall
 - Surface Condition
-

34. Result

Recommended Pressure

FRONT

28 PSI

1.93 BAR

REAR

31 PSI

2.14 BAR

Toggle:

PSI | BAR

35. Result Explanation

Beispiel:

Optimized for your 45 mm tubeless setup and this route's surface mix.

Route Surface

42 % Paved

51 % Gravel

7 % Trail

Why this pressure?

Optional aufklappbar:

- System Weight
- Tire Width
- Front/Rear Load
- Ride Type
- Weight Distribution
- Terrain Mix
- Tire Setup

Später:

- Surface Condition
 - Weather
-

36. Product Transparency

RouteRider soll keine falsche Präzision vermitteln.

The recommendation is a starting pressure based on your setup and available route data.
Tire construction, rim width, riding style and personal preference can affect your ideal pressure.

Finale Formulierung noch offen.

37. UX Architecture

01 Route

Upload GPX

↓

02 Surface

Analyze Route + Surface

↓

03 Setup

Bike + Rider + Ride Type

↓

04 Pressure

Recommendation

Keine komplexe Navigation.

Kein Dashboard.

Kein Account.

38. Design Direction

RouteRider soll:

- modern
- hochwertig
- technisch
- outdoor-orientiert
- reduziert

wirken.

Nicht:

- verspielt
- überladen
- wissenschaftlicher Calculator
- generisches AI Dashboard

39. Technical Architecture

``text id="scpmrt" GPX File | ▼ GPX Parser | ▼ Route Geometry | ▼ Map Matching | ▼ Matched OSM Ways | ▼ OSM Attributes | |—— surface |—— smoothness |—— tracktype |—— highway | ▼ Surface Classification | |—— Paved |—— Gravel |—— Trail |—— Unknown | ▼ Distance-weighted Surface Mix | ▼ Surface Confidence | ▼ Bike Setup | ▼ Pressure Engine | ▼ Front / Rear Recommendation

Future:

```
``text id="4xx7h1"
Weather Data
  |
  ▼
Surface Condition
  |
  ▼
Pressure Engine
```

40. Technical Stack

Framework: Next.js

Frontend: React + TypeScript

Architecture: Next.js App Router

Map: Leaflet

Map Data: OpenStreetMap
GPX: GPX → GeoJSON
Surface Data: OpenStreetMap
OSM Query: initially Overpass or selected alternative
Map Matching: ● to be decided
External API Handling: Next.js Server Routes
Hosting: Vercel
Database: None for MVP
Authentication: None
Accounts: None

41. Infrastructure Principle

Public OpenStreetMap/Overpass infrastructure can be used for development and early testing.

The architecture must not assume that free public endpoints are the permanent production infrastructure.

Later options can include:

- dedicated OSM provider
- hosted Map Matching
- own OSM infrastructure
- caching

This is not an MVP blocker.

42. MVP Scope

Included

- ✓ GPX Upload
- ✓ GPX Parsing
- ✓ Route Map
- ✓ Distance
- ✓ Elevation where reliable
- ✓ Map Matching
- ✓ OSM Surface Analysis
- ✓ Paved / Gravel / Trail / Unknown
- ✓ Surface Coverage
- ✓ Confidence
- ✓ Manual Correction
- ✓ Rider Weight
- ✓ Bike Weight
- ✓ Luggage Weight
- ✓ Tire Width

- ✓ Tube / Tubeless Input
- ✓ Ride Type
- ✓ Weight Distribution
- ✓ Berto Base Pressure
- ✓ Terrain Modifier
- ✓ Front / Rear Pressure
- ✓ PSI / BAR
- ✓ Result Explanation

Explicitly Not Included

- ✗ Weather Pressure Adjustment
 - ✗ Detailed Surface Roughness
 - ✗ Smooth/Rough Gravel
 - ✗ User Accounts
 - ✗ Database
 - ✗ Strava
 - ✗ Komoot Integration
 - ✗ Route Planning
 - ✗ Navigation
 - ✗ POIs
 - ✗ Fueling
 - ✗ Social Features
 - ✗ Native Apps
-

43. Definition of Done — V0.1

Der MVP ist funktionierend, wenn ein Nutzer:

1. GPX hochladen kann
 2. Route auf Karte sieht
 3. Routendaten erhält
 4. Route zuverlässig gegen OSM gematched wird
 5. automatisch einen Surface Mix erhält
 6. Surface Coverage und Confidence sieht
 7. bei Low Confidence gewarnt wird
 8. Surface Mix korrigieren kann
 9. Bike-Setup eingibt
 10. Ride Type auswählt
 11. Weight Distribution erhält und verändern kann
 12. Front/Rear Pressure erhält
 13. PSI/BAR wechseln kann
 14. Recommendation nachvollziehen kann
 15. den Flow mobil verwenden kann
-

44. Development Phases

Phase 0

Project Setup

Phase 1

GPX Upload + Parsing

STOP → Test

Phase 2

Route Map + Route Information

STOP → Test

Phase 3

Map Matching Prototype

STOP → Test with multiple real GPX routes

Phase 4

OSM Surface Analysis

STOP → Validate against known routes

Phase 5

Confidence + Manual Correction

STOP → Test

Phase 6

Bike Setup + Ride Type

STOP → Test

Phase 7

Pressure Engine

STOP → Calibration Tests

Phase 8

Result Screen

STOP → UX Test

Phase 9

Mobile Optimization

Phase 10

Deployment

Nach jeder funktionierenden Phase:

Git Commit.

45. Development Rule

Claude Code soll bei Unsicherheit über:

- Product Logic
- Pressure Calculation
- Surface Classification
- Map Matching Architecture
- UX
- External Services

keine größere Produktentscheidung eigenständig treffen.

Stattdessen:

Stop and ask for clarification.

46. Resolved Decisions

● Warning only at Low Confidence (<60 %)

● compacted = Gravel

● Smooth/Rough Gravel comes later

- Weight Distribution mandatory
 - Performance/Race = 45/55 initial suggestion
 - Gravel/Endurance = 47/53 initial suggestion
 - Bikepacking/Loaded = 40/60 initial suggestion
 - Berto-based Base Pressure Model
 - Paved Modifier = 1.00
 - Gravel Modifier = 0.90
 - Trail Modifier = 0.80
 - Distance-weighted Surface Mix
 - Unknown Surface normalization + manual correction
 - GPX and OSM are separate data sources connected through Map Matching
 - Additional OSM attributes should be retained for future development
 - Weather should later influence expected Surface Condition
 - Current + recent precipitation are potentially relevant
 - Weather is not part of V0.1
-

47. Critical Open Decisions

● D4 — Tubeless Logic

Determine whether Tubeless:

- directly changes recommended pressure
 - primarily changes safe pressure limits
 - or requires a combination
-

● D5 — Map Matching Architecture

Compare:

- Own OSM Geometry Matching

- Existing Map Matching Services

Select solution for MVP.

● D6 — Safety Pressure Bounds

Determine how RouteRider prevents recommendations outside safe tire/rim limits.

● D7 — Surface Inference Rules

Define how:

tracktype

highway

smoothness

can supplement missing surface=*

48. Future Backlog

Surface Intelligence

- Smooth / Rough Gravel
- Surface Roughness Score
- Better Confidence Model
- Critical Rough Sections

Weather Intelligence

- Current precipitation
- Previous 24h rainfall
- Previous 48h rainfall
- Expected rainfall during ride
- Dry / Damp / Wet / Muddy Surface Prediction

Bike Intelligence

- Rim Internal Width
- Tire Casing
- Tire Model
- Bike Geometry
- Luggage Position

Route Intelligence

- Rough Section Priority
 - Route Planning
 - Weather-aware Routing
 - POIs
 - Water
 - Fueling
-

49. Current Priority

Die Kernfrage bleibt:

Wie bauen wir den bestmöglichen route-aware Tire Pressure Calculator?

Dafür müssen zunächst drei technische bzw. methodische Unsicherheiten beseitigt werden:

1. Tubeless Logic

2. Map Matching

3. Safe Pressure Bounds

Danach ist die fachliche Grundlage ausreichend definiert, um den eigentlichen Claude-Code-Build strukturiert zu starten.

50. Living Document Rules

Dieses Dokument ist **nicht final**.

Status:

 **Open / must resolve**

 **Draft / needs refinement**

 **Decided**

Neue Ideen werden zunächst bewertet und nicht automatisch Bestandteil des MVP.

Das Dokument soll jederzeit beantworten:

Was wissen wir?

Was nehmen wir aktuell an?

Was ist entschieden?

Was ist noch offen?

Was bauen wir als Nächstes?